Conditional noexcept specifiers in compound requirements

Document #:	P3822R1 [Latest] [Status]
Date:	2026-02-23
Project:	Programming Language C++
Audience:	EWG

Reply-to: Viacheslav Luchkin <someone12469@gmail.com>
Gašper Ažman <gasper.azman@gmail.com>

Contents

§ 1 Introduction

This paper extends compound requirements to allow noexcept specifiers to be applied conditionally. The proposed syntax for this is `requires { { expression } noexcept(constant-expression) -> return-type-constraint; }`.

§ 2 Revision history

Revision 1 adds implementation and history information, adds another example and replaces *condition* with *constant-expression* to avoid ambiguity with `if`/`while`/`switch` conditions.

§ 3 Motivation

Requires-expressions can be used to assert that an expression is non-throwing, but do not provide a way to do so conditionally, which is sometimes needed in generic programming. Achieving this now usually requires code duplication.

The lack of support for this syntax is also inconsistent with function declarations, which have had conditional noexcept specifiers since C++11.

Motivational example:

Before

```
template<typename F, bool noexc>
concept invocable = noexc
  ? requires(F f) { { f() } noexcept;
        }
  : requires(F f) { f(); };
template<bool noexc>
struct callable_ref {
  callable_ref(invocable<noexc>
        auto&& fn);
  [...]
};
```

After

```
template<typename F, bool noexc>
concept invocable = requires(F f) {
  { f() } noexcept(noexc);
};
template<bool noexc>
struct callable_ref {
  callable_ref(invocable<noexc>
        auto&& fn);
  [...]
};
```

§ 4 History

The current syntax for unconditional noexcept specifiers in requirements appears to originate from [N3701]. Some searching resulted in no evidence that making them conditional was discussed or rejected due to encountering issues.

§ 5 Proposal

Redefine the syntax for compound requirements as follows:

compound-requirement:
  `{` *expression* `}` ~~`noexcept`~~$_{\text{opt}}$ *noexcept-specifier*$_{opt}$ *return-type-requirement*$_{opt}$ `;`

noexcept-specifier:
  `noexcept(`*constant-expression*`)`
  `noexcept`

return-type-requirement:
  `->` *type-constraint*

Where the *constant-expression*, if supplied, shall be a contextually converted constant expression of type `bool`. The *noexcept-specifier* `noexcept` without a *constant-expression* is

equivalent to `noexcept(true)`.

If *constant-expression* evaluates to `true` and *expression* is a potentially-throwing expression, the requirement is not satisfied. If *constant-expression* evaluates to `false` or the *noexcept-specifier* is absent, *expression* may be potentially-throwing.

If the conversion of *constant-expression* to `bool` fails, the requirement is not satisfied. This is not a hard error, because requirements are often used in overload resolution and the expected behavior for an unexpected input type is that other overloads are still considered.

§ 6 Implementation experience

Yuxuan Chen provided an implementation of this proposal in a Clang fork, which should soon be available on Compiler Explorer at https://godbolt.org/z/saYoeWPM9. Most existing logic for working with function exception specifications was reusable for compound requirements.

§ 7 Proposed wording

Recall that the grammar for *noexcept-specifier* is defined in section 14.5 [except.spec] as follows:

> *noexcept-specifier:*
> `noexcept(`*constant-expression*`)`
> `noexcept`

Alter section 7.5.8.4 [expr.prim.req.compound] as follows:

> *compound-requirement:*
> `{` *expression* `}` ~~`noexcept`$_{opt}$~~ *noexcept-specifier*$_{opt}$ *return-type-requirement*$_{opt}$ `;`
> *return-type-requirement:*
> `->` *type-constraint*

1 A *compound-requirement* asserts properties of the *expression* *E*. The *expression* is an unevaluated operand. Substitution of template arguments (if any) and verification of semantic properties proceed in the following order:

(1.1) — Substitution of template arguments (if any) into the *expression* is performed.

(1.2) — If the `noexcept` specifier is present, *E* shall not be a potentially-throwing expression ([except.spec])

(1.2) — If the *noexcept-specifier* ([except.spec]) is present, then:

(1.2.1) — Substitution of template arguments (if any) into its associated *constant-expression* is performed. The *noexcept-specifier* `noexcept` without a *constant-expression* is equivalent to `noexcept(true)`.

(1.2.2) — The *constant-expression* shall be a contextually converted constant expression of type `bool` ([expr.const]). If the *constant-expression* is true, *E* shall not be a potentially-throwing expression ([except.spec]).

Bump feature-test macro `__cpp_concepts` in section 15.12 [cpp.predefined]:

```
  __cpp_char8_t 202207L
- __cpp_concepts 202002L
+ __cpp_concepts 20XXXXL
  __cpp_conditional_explicit 201806L
```

§ 8 Appendix

A more complete example using this to implement type erasure (link):

```
#include <concepts>
#include <memory>
#include <type_traits>
#include <utility>

template <typename R, typename... Args>
struct vtbl { virtual auto call(Args&&...) -> R = 0; virtual ~vtbl() noex-
        cept {}; };

template <typename T, typename R, typename...Args>
struct impl : vtbl<R, Args...> {
    impl(auto&& y) : x{std::forward<decltype(y)>(y)} {}
     auto call(Args&&... xs) -> R override { return x(static_cast<Args&&>
        (xs)...); }

    T x;
};

template <typename F>
struct any_f;

template <typename X, bool noexc, typename R, typename...Args>
concept invocable_r = requires (X x, Args... xs) {
     { x(static_cast<Args>(xs)...) } noexcept(noexc) -> std::convertible_-
        to<R>;
};

template <typename X, typename T>
concept not_same = !std::same_as<std::decay_t<X>, T>;

template <typename R, typename... Args, bool noexc>
struct any_f<R(Args...) noexcept(noexc)> {
    template <not_same<any_f> T>
```

```
    any_f(T&& x)
        requires invocable_r<T&, noexc, R, Args...>
        : _f(new impl<std::decay_t<T>, R, Args...>(std::forward<T>(x))) {}

    // standard type erasure
         auto  operator()(std::convertible_to<Args>  auto&&...  xs)  noex-
        cept(noexc) -> R {
        return _f->call(std::forward<decltype(xs)>(xs)...);
    }

    std::unique_ptr<vtbl<R, Args...>> _f;
};

int main() {
    any_f<int(long, long) noexcept> x([](long, long) noexcept -> int { re-
        turn 2; });
}
```

Similar code also triggers some bugged-looking diagnostics in GCC 15.2 (link).

§ 9 References

[N3701] A. Sutton, B. Stroustrup, G. Dos Reis. 2013-06-28. Concepts Lite.
https://wg21.link/n3701